

SP.



Saved, along with this attempt's result. I'll write from contact@szymonpaluch.com when the next tests and materials are ready. Check your spam folder too, it's an ordinary email.

PASS

955
SCALED SCORE

57 / 60 correct (95%) · pass line 720 ≈ 42/60 correct

Active time used: 35m 0s of 120m (timer paused during feedback)

[Retake exam](#)

PERFORMANCE BY DOMAIN

D1 · Agentic Architecture & Orchestration	16/16 · 100%
D2 · Tool Design & MCP Integration	10/11 · 91%
D3 · Claude Code Configuration & Workflows	11/12 · 92%
D4 · Prompt Engineering & Structured Output	11/12 · 92%
D5 · Context Management & Reliability	9/9 · 100%

Review & explanations

All

Incorrect

Flagged

Q1

D1 · Agentic

Task 1.1

Correct

Customer Support Resolution Agent

that ends the conversation whenever the assistant's text matches a closing phrase like "let me know if there's anything else." Production logs show that in 11% of sessions the loop terminates while a `tool_use` block is still pending, so the refund is never actually executed. What is the correct way to drive the agentic loop?

- A

Terminate only when `stop_reason` is `end_turn`; whenever `stop_reason` is `tool_use`, execute the requested tool, append the `tool_result` to the message list, and continue the loop.

✓ correct
- B Expand the regex allowlist of closing phrases so it recognizes every polite sign-off the model might emit before stopping.
- C Add a max-iteration cap and stop the loop once the assistant produces two consecutive text-only responses.
- D Run a lightweight intent classifier on each assistant message to decide whether the turn represents a genuine completion.

Why A is correct. The agentic loop must be driven by the API's `stop_reason` signal, not by parsing natural-language content. `tool_use` means the model wants a tool run, so you execute it, append the `tool_result`, and iterate; `end_turn` is the only correct termination signal. Matching closing phrases (B) is exactly the brittle text-parsing that caused the bug and will still misfire. An iteration cap plus consecutive text heuristic (C) can truncate legitimate multi-step work and still ignores `stop_reason`. A classifier (D) is over-engineering that reintroduces natural-language inference where a deterministic protocol field already gives the answer.

moment of escalation to fix this?

- A Attach the full untrimmed transcript of the conversation so the human has complete information.
- B Enable extended thinking on the escalation turn so the model reasons longer before handing off.
- C Escalate earlier in ambiguous cases so humans receive tickets before much conversation has accumulated.

D Produce a structured handoff summary — the customer's issue, verified identifiers, tool results and account state gathered, actions already attempted, and the specific recommended next step.

✓ correct

Why D is correct. Effective human handoff requires a structured summary that front-loads exactly what the human needs: the problem, confirmed identifiers, what was tried, and a recommendation. That is what removes the re-reading burden and shortens handle time. Dumping the full transcript (A) is the current failure mode — it forces the human to reconstruct the case themselves. Extended thinking (B) improves the model's own reasoning but produces no artifact for the human. Escalating earlier (C) changes when tickets arrive, not the missing information problem, and would likely lower first-contact resolution.

Q3 D1 · Agentic Task 1.5

Correct

Customer Support Resolution Agent

Company policy states that any refund above \$500 must be approved by a manager rather than auto-issued. The support agent, however, has autonomously called `process_refund` for amounts over \$500 in several audited cases. What is the

SP.



with an amount over \$500 and redirects the flow to `escalate_to_human`.

✓ correct

- B Add a line to the system prompt instructing the agent to always escalate refunds over \$500 instead of processing them.
- C Remove `process_refund` from the agent's toolset entirely so every refund is handled by a human.
- D Fine-tune the model on historical tickets so it learns the \$500 threshold from past manager approvals.

Why A is correct. A policy violation that must never occur calls for an interception (PreToolUse) hook that deterministically blocks the disallowed `process_refund` call and redirects to escalation. The hook enforces the threshold regardless of what the model decides. A prompt instruction (B) is advisory and is precisely what the audited violations bypassed. Removing the tool (C) throws away all legitimate sub-\$500 automation and tanks first-contact resolution. Fine-tuning (D) is heavy, slow, and still probabilistic — it cannot promise the hard \$500 boundary is never crossed.

Q4

D1 · Agentic

Task 1.5

Correct

Customer Support Resolution Agent

`get_customer` and `lookup_order` call two different backend systems that return dates in different formats (ISO vs. US) and amounts with inconsistent currency notation. Logs show the model occasionally misreads these heterogeneous fields, e.g., treating a refund amount in the wrong units. What is the most robust remedy?

- B** Add a PostToolUse hook that normalizes each tool's raw output into one canonical schema before it is appended to the context.

✓ correct

- c Increase the context window so the model has more room to keep the varied formats straight.
- D Spin up a dedicated formatting subagent that the coordinator calls after every backend lookup.

Why B is correct. A PostToolUse hook is the designed mechanism for normalizing heterogeneous tool output into a consistent schema before it ever reaches the model, eliminating the ambiguity deterministically. Asking the model to normalize in-prompt (A) leaves the same error-prone inference in place — the source of the current misreads. A larger context window (C) addresses volume, not format inconsistency, and does nothing for unit confusion. A separate formatting subagent (D) adds a full round-trip and orchestration overhead for a transformation a hook performs inline and reliably.

Q5

D1 · Agentic

Task 1.6

Correct

Customer Support Resolution Agent

The coordinator has already run `get_customer` and confirmed the account, then spawns subagents to check order history and refund eligibility in parallel. Those subagents return results that contradict the confirmed account — some even re-run `get_customer` and pick a different customer match. What is the correct fix?

- A Enlarge the shared context window so the subagents can see the coordinator's earlier turns.

- c Instruct each subagent to call `get_customer` itself at the start so it independently establishes the account.

D Include the coordinator's already-verified findings — the confirmed customer record and identifiers — explicitly in each subagent's Task prompt, since subagents do not inherit the coordinator's conversation context.

✓ correct

Why D is correct. Subagents run with their own fresh context and do not inherit the coordinator's conversation, so any prior findings they must use have to be passed explicitly in the Task prompt. Handing each subagent the confirmed customer record eliminates the contradictions and the redundant lookups. Enlarging a "shared" context window (A) misdescribes the architecture — there is no shared window to grow across the hub-and-spoke boundary. There is no SDK mechanism that auto-injects a global variable into subagents (B); it is a plausible-sounding non-feature. Having each subagent re-run `get_customer` (C) is exactly what produced the divergent matches and duplicates work.

Q6

D2 · Tools & MCP

Task 2.1

Correct

Customer Support Resolution Agent

`process_refund` handles both full order refunds and small partial goodwill credits. Logs show the agent issues full refunds when only a partial credit was warranted in 17% of cases because it cannot tell from the single tool which behavior it is invoking. What is the best fix?

- A Raise the agent's context window so the entire refund policy document fits alongside the conversation.

B Split `process_refund` into two tools, `process_full_refund` and `process_partial_credit`, each with a distinct description and explicit boundary

- c Add three few-shot examples of partial-credit conversations to the system prompt while keeping the single tool.
- d Set tool_choice to "any" so the model is compelled to select a tool on every refund turn.

Why B is correct. When one tool conflates two distinct behaviors, the reliable fix is to split it into separately named tools whose descriptions encode clear boundaries, making the choice explicit at selection time. Option A adds context but leaves the overloaded tool ambiguous, so the same misclassification persists. Option C helps at the margins but few-shot prompt examples cannot substitute for a tool interface that still exposes one undifferentiated action. Option D only guarantees that some tool is called, not that the correct refund behavior is chosen, and does nothing about the underlying overloading.

Q7 D2 • Tools & MCP Task 2.2

Correct

Customer Support Resolution Agent

When the customer database is briefly unreachable, get_customer returns a plain empty result, and the agent tells the customer "I can't find any account with that information" and closes the case. Real accounts are being turned away during transient outages. What should the MCP tool return instead?

- A Automatically retry get_customer in a tight loop until the database responds, however long that takes.
- B Increase the get_customer connection timeout so slow database responses are no longer cut off.

C A structured error with an errorCategory (e.g. transient) and isRetryable metadata that lets the agent distinguish an access failure from a valid empty

- D Treat every non-successful `get_customer` response as an escalation trigger and route all of them to `escalate_to_human`.

Why C is correct. The root cause is that an access failure is being conveyed as a valid empty result, so the agent draws the wrong conclusion; the tool must emit structured error metadata (`errorCategory`, `isRetryable`) that separates 'lookup failed' from 'no such customer.' Option A retries blindly with no bound and still lacks the semantics to tell the two conditions apart. Option B may reduce some timeouts but does not fix the core ambiguity when a failure does occur. Option D over-escalates by treating validation and business outcomes identically to infrastructure faults, defeating the resolution target and burying real transient errors.

Q8

D2 · Tools & MCP

Task 2.2

Correct

Customer Support Resolution Agent

For refunds above the agent's \$500 authority, `process_refund` correctly declines, but it returns a generic `isError` response. Logs show the agent then retries the same \$640 refund three times before eventually giving a confusing reply. How should the tool signal this condition?

- A Have the agent retry with exponential backoff, since repeated attempts eventually succeed for most declined refunds.
- B Raise the agent's refund authority limit above \$640 so the call no longer fails.
- C Instruct the agent in the system prompt to never retry a refund more than once.

D Return a structured error with `errorCategory` "business" and `isRetryable` set to

Why D is correct. A policy-limit rejection is a business error that is deterministically non-retryable; encoding `errorCategory "business"` with `isRetryable false` tells the agent that retrying is pointless and that escalation is the correct next step. Option A applies retry logic meant for transient faults to a permanent business rule, wasting calls and never succeeding. Option B changes policy to dodge the control rather than handling the boundary the business intends to enforce. Option C is a prompt-only patch that leaves the error semantically indistinguishable from a retryable one, so behavior stays fragile compared with programmatic metadata the agent can act on reliably.

Q9

D5 • Context

Task 5.1

Correct

Customer Support Resolution Agent

In longer billing-dispute conversations, the agent uses progressive summarization to keep the history compact. Testers find that after several turns it quotes the wrong refund amount and an incorrect order date, even though those values appeared earlier. What is the best remedy?

- ☒ A Maintain a persistent 'case facts' block (order id, amount, dates, customer expectation) stored outside the summarized history and re-injected verbatim each turn.
✓ correct
- ☐ B Switch to a model with a larger context window so no summarization is needed.
- ☐ C Call `get_customer` and `lookup_order` again on every turn to refetch the numbers.
- ☐ D Enable extended thinking so the model reasons more carefully before quoting

Why A is correct. Progressive summarization compresses away precise numbers, dates, and stated expectations, so the fix is to keep those exact values in a persistent case-facts block that lives outside the summarized transcript and is re-supplied unchanged. Option B only delays the same information loss once conversations grow, and does not protect the specific fields. Option C is wasteful and can still be re-summarized away, plus it re-incurs latency and error surface each turn. Option D improves reasoning but cannot recover facts that summarization has already discarded from context.

Q10

D5 · Context

Task 5.2

Correct

Customer Support Resolution Agent

A customer gives only the name "Maria Garcia." `get_customer` returns three distinct matching accounts. Logs show the agent proceeds with the most recently active match, and in several cases it has processed refunds against the wrong customer's order. What is the correct behavior?

- A Continue with the most recently active of the three accounts, since it is the most likely current customer.
- B Pick the match with the highest fuzzy name-similarity score and proceed.
- C Escalate every multiple-match case straight to a human agent.



Ask the customer for an additional identifier (order number, email, or ZIP) to disambiguate before taking any account action.

✓ correct

Why D is correct. When multiple customer records match, the agent must not guess; requesting an additional identifier resolves the ambiguity and prevents acting on the wrong account. Option A guesses on recency and is exactly the

undermining the first-contact-resolution target when disambiguation is straightforward.

Q11

D3 · Claude Code

Task 3.1

Correct

Code Generation with Claude Code

Your team standardized naming and error-handling conventions by adding them to `~/.claude/CLAUDE.md` on the tech lead's machine. Everything works for the lead, but three newly onboarded engineers who cloned the repo report that Claude Code ignores the conventions entirely, even though ``git status`` shows a clean checkout. What is the most effective fix?

- A Add an `@import` to the repo-root `CLAUDE.md` pointing at `~/.claude/CLAUDE.md` so the personal file is pulled in for everyone
- B Have each engineer run `/memory` after cloning to force-reload the user-scoped memory files
- C** Move the conventions into the project-scoped `.claude/CLAUDE.md` (or repo root) so they are committed to version control and load for every clone
✓ correct
- D Convert the conventions into a skill under `.claude/skills/` so they activate on demand for all users

Why C is correct. The `CLAUDE.md` hierarchy loads user-scoped instructions from `~/.claude/CLAUDE.md`, which is personal to a machine and is NOT shared through version control — the classic 'new teammate didn't get the instructions' symptom. Committing the conventions to the project-scoped `.claude/CLAUDE.md` (or repo root) makes them load for every clone, so C is correct. An `@import` to `~/.claude/CLAUDE.md` fails because that file doesn't exist on the teammates' machines. /

SP.



Code Generation with Claude Code

Your team's `/investigate-flaky` skill greps the entire test suite, reads dozens of files, and prints verbose exploration before it answers. Developers notice that after running it, the main session is cluttered with raw output and Claude's subsequent answers in that session degrade in quality. What is the best fix?

- A Add a step to `CLAUDE.md` instructing Claude to summarize the skill's output before continuing

- B** Set `context:fork` in the skill's `SKILL.md` frontmatter so its verbose exploration runs in an isolated context and only the result returns to the main session

✓ correct

- C Increase the context window so the accumulated exploration output no longer causes degradation
- D Restrict the skill with an `allowed-tools` list so it reads fewer files during exploration

Why B is correct. Skills support `context:fork` in their `SKILL.md` frontmatter precisely to isolate verbose, exploratory work in a separate context and return only the distilled result, keeping the main session clean — so B is correct. A `CLAUDE.md` instruction to summarize is a prompt-only measure that runs after the pollution already occurred and is unreliable. A larger context window does not fix attention dilution from irrelevant accumulated output; it just delays it. `allowed-tools` restricts which tools the skill may use but does not stop verbose output from landing in the main context, and it would also cripple the investigation the skill exists to perform.

Q13

D3 · Claude Code

Task 3.2

Correct

Code Generation with Claude Code

refactoring sessions. The team still wants engineers to be able to run `/dep-audit` explicitly. What is the most effective fix?

- A Call `/compact` automatically at the end of the skill so the audit output is condensed before work continues
- B Set `disable-model-invocation: true` in the skill's SKILL.md frontmatter so the skill runs only when a user explicitly invokes `/dep-audit`**
✓ correct
- C Delete the skill's `description` frontmatter so Claude can no longer match it to conversation topics
- D Move the audit instructions into CLAUDE.md so they load once at startup instead of triggering mid-session

Why B is correct. `disable-model-invocation: true` is the frontmatter switch designed for exactly this: it removes the skill from Claude's automatic invocation while keeping explicit `/dep-audit` slash invocation available, so release engineers keep the workflow and refactoring sessions stay clean. `/compact` (A) reacts only after the 4,000 lines have already polluted and degraded the session, and compaction can discard needed detail. Deleting the description (C) is a hack — the description also documents the skill for humans and tooling, and gutting metadata to suppress triggering is not the supported mechanism. Moving the instructions into CLAUDE.md (D) changes where guidance loads but makes the verbose audit part of every session's baseline context, worsening rather than fixing the problem.

Q14

D3 · Claude Code

Task 3.2

Correct

Code Generation with Claude Code

A shared `generate-docs` skill is supposed to read source and write Markdown into

source?

- A Add a stronger system-prompt instruction forbidding edits to any file outside ``docs/``
- B Require the skill to run in plan mode every time so a human approves each action
- C** Declare ``allowed-tools`` in the skill's SKILL.md frontmatter, restricting it to Read/Glob/Write so the harness blocks Edit and Bash entirely
✓ correct
- D Add a global PreToolUse hook that inspects every Edit across all sessions and rejects ones outside ``docs/``

Why C is correct. When deterministic tool restriction is required, prompt text is insufficient because the model can still choose a forbidden tool; the ``allowed-tools`` frontmatter enforces the restriction at the harness level, scoped exactly to this skill. Stronger prose (option A) is still an instruction the model may ignore, which is the observed failure. Plan mode adds friction to every run and relies on the human catching the bad action rather than preventing it. A global PreToolUse hook is the right idea at the wrong scope — it would police every session's edits when the problem is confined to one skill, and ``allowed-tools`` is the purpose-built, skill-scoped mechanism.

Q15

D3 · Claude Code

Task 3.2

Correct

Code Generation with Claude Code

A shared ``/scaffold-component`` skill expects a component-name argument, but telemetry shows teammates invoke it with no argument or a malformed one about a third of the time, producing empty or misnamed scaffolds. What is the most

expected argument and format are shown at the invocation prompt

✓ correct

- B Document the correct usage in the project CLAUDE.md so it is always loaded into context
- C Rename the skill to `/scaffold-component-with-name` so the required input is implied by the name
- D Have the skill body instruct Claude to ask the user for the name whenever it is missing

Why A is correct. `argument-hint` is the frontmatter field designed precisely to surface the expected argument and format at the moment of invocation, guiding the caller before a malformed run happens. Documenting usage in CLAUDE.md (option B) helps the model but does not present the hint to the human at invocation time, where the mistake occurs. Renaming (option C) is a cosmetic nudge that neither shows the format nor prevents empty calls. A body instruction to ask when missing (option D) is a runtime fallback that adds a round-trip and still does not communicate the expected format up front — the direct, purpose-built fix is `argument-hint`.

Q16

D3 · Claude Code

Task 3.5

Incorrect

Code Generation with Claude Code

A developer needs three changes that all touch the shared `PaymentStatus` enum and the several functions that switch on it. They sent each change as a separate sequential message; after the third message the project no longer compiles because the earlier edits did not anticipate the later ones. What is the best way to structure the request?

X your answer

- B Enable plan mode and approve the three changes one at a time in sequence
- C Keep them sequential but ask Claude to re-read the enum before each message

D Batch the three interacting changes into a single message so Claude edits the enum and all dependent switch sites together with full mutual context

✓ correct

Why D is correct. Fixes that interact — here all three touch one enum and its dependent switches — should be batched into a single message so the model reasons about them jointly and keeps the code internally consistent; only independent changes benefit from being sequenced separately. Separate fresh sessions (option A) make the interaction worse by discarding shared context each time. Plan mode approvals (option B) still process the edits one at a time, reproducing the same ordering problem. Re-reading the enum each message (option C) treats a coordination problem as a memory problem; the real fix is to let the model see all three interacting edits at once.

Q17

D4 · Prompting

Task 4.1

Correct

Code Generation with Claude Code

Asking Claude to "make the input validation more robust and idiomatic" yields inconsistent output across runs: sometimes it adds zod, sometimes hand-written checks, sometimes nothing meaningful. The team wants the validation style to come out consistent. What is the most effective refinement?

- A Expand the request into a longer, more detailed prose description of what "robust" and "idiomatic" mean

✓ correct

- c Fine-tune a model on the team's existing validation code so it internalizes the house style
- D Add a standing CLAUDE.md rule stating that validation should always be robust and idiomatic

Why B is correct. For ambiguous, style-sensitive tasks, concrete input/output examples communicate the target far more reliably than prose and generalize across cases — a few examples pin down the exact pattern the vague adjectives left open. More detailed prose (option A) still leaves "robust" and "idiomatic" open to interpretation, which is the source of the variance. Fine-tuning (option C) is heavy over-engineering for what a few examples in the prompt solve. A CLAUDE.md rule (option D) restates the same vague adjectives as an always-loaded instruction without demonstrating the concrete format, so the inconsistency persists.

Q18

D5 · Context

Task 5.4

Correct

Code Generation with Claude Code

During a 200-file framework migration carried out in one long session, logs show Claude re-migrating files it already converted and skipping others as the conversation grows. What technique best keeps the migration on track?

- A Switch to a larger-context model so the whole file list stays in the window
- B Run ``/compact`` frequently to keep the history short

C

Maintain a scratchpad manifest file that records migrated vs pending files and update it after each file, so progress survives context growth

D Do the entire migration in a single large message to avoid state loss between turns

Why C is correct. For large, long-running tasks the durable pattern is an external scratchpad or manifest that tracks state on disk, so progress is authoritative regardless of what falls out of the conversation window. A larger window (option A) still degrades over a 200-file span and does not give a reliable record of what is done. `/compact` (option B) shrinks history but can summarize away exactly the per-file progress you need, and it does not create a checkable record. A single massive message (option D) is impractical at this scale and still offers no crash-safe record of completed work — the manifest provides both tracking and recovery.

Q19

D5 · Context

Task 5.4

Correct

Code Generation with Claude Code

To rename a widely-used internal API, Claude must first locate every call site across a large monorepo. Doing this discovery inline reads dozens of files and floods the main session with search output before any edits begin, and the later edits then start to wander. What is the best approach?

A Delegate the discovery to an Explore subagent that searches the monorepo and returns only a summary of call sites, keeping the main context focused on the edits

✓ correct

- B Read every candidate file into the main session first so all call sites are visible before editing
- C Run `/compact` after each search so the accumulated output is repeatedly condensed

Why A is correct. Verbose discovery over a large codebase should be delegated to a subagent so the exploration happens in an isolated context and only a distilled summary of call sites returns — the main session stays focused on performing the rename. Reading every file into the main session (option B) is exactly the flooding behavior causing the edits to wander. Repeated `/compact`` (option C) is a lossy patch applied after the noise already accumulated and interrupts the flow. A bigger window (option D) still suffers attention dilution from the raw search output; isolating the discovery is what preserves the working context.

Q20

D5 · Context

Task 5.4

Correct

Code Generation with Claude Code

Midway through a coherent multi-hour refactor, the session approaches its context limit. The history is full of superseded intermediate diffs, but the current task and goal are unchanged and the latest file states are still valid. What should the developer do to keep going without losing the thread?

- A Start a brand-new session with no prior context and re-describe the task
- B Fork the session to branch off a divergent line of work
- C Request a larger context window and continue as-is

D Run `/compact`` to condense the accumulated history into a summary while continuing the same task

✓ correct

Why D is correct. When the task is a single coherent thread and only the accumulated intermediate history is bloating the context, `/compact`` condenses

is for pursuing a divergent branch, not for continuing one coherent line. A larger window (option C) postpones the limit without addressing the superseded-diff clutter; compaction removes the dead weight while preserving the goal.

Q21

D5 · Context

Task 5.1

Correct

Code Generation with Claude Code

In a long debugging session Claude repeatedly runs the full test suite; each run dumps roughly 2,000 lines of passing-test noise into the context. Over time Claude starts overlooking the failing-test details and the constraints stated at the start of the session. What is the best fix?

- A Switch to a larger-context model so all the test output fits comfortably
- B Trim each test run's output to the relevant failing-test lines before it accumulates, and keep the key constraints in a summary placed at the top of the context**
✓ correct
- C Enable extended thinking so Claude reasons harder over the full accumulated output
- D Instruct Claude to re-read the entire session history at the start of every turn

Why B is correct. The failure is context degradation from unbounded verbose tool output plus lost-in-the-middle placement of the key constraints; the fix is to trim tool outputs to the relevant fields before they accumulate and to keep the critical summary up front where attention is strongest. A larger window (option A) still lets 2,000 lines per run bury the failing details and dilute attention — capacity is not the problem, signal-to-noise is. Extended thinking (option C) makes the model reason harder over the same noisy context without removing the noise. Re-reading the

Q22

D1 · Agentic

Task 1.3

Correct

Multi-Agent Research System

The synthesis subagent's reports read as generic overviews: in 30% of runs they omit the specific figures and named sources the web-search subagent surfaced. Tracing shows the Task prompt sent to the synthesis subagent contains only the report topic and formatting instructions. What is the best fix?

- A Increase the coordinator's context window so more findings stay resident across delegations
- B Have the coordinator embed the web-search and document-analysis findings verbatim in the synthesis subagent's Task prompt**
✓ correct
- C Add a 'recall the earlier research' instruction to the synthesis subagent's system prompt
- D Give the synthesis subagent its own web-search tool so it can re-gather the missing figures

Why B is correct. Subagents do not inherit the coordinator's conversation context; in a hub-and-spoke topology the coordinator must pass prior findings explicitly in the Task prompt, so embedding the web-search and document-analysis results verbatim is the fix. A larger context window (A) does not cross the process boundary into a freshly spawned subagent. Instructing it to 'recall' earlier research (C) fails because the subagent starts with no prior turns to recall. Giving it its own search tool (D) is wrong scope and over-engineering — it duplicates work and diverges from the evidence already gathered.

Multi-Agent Research System

The coordinator dispatches three independent regional web-search subagents (EU, US, APAC), but issues each Task call only after the previous subagent returns. End-to-end research latency is about 3x the slowest single search. What change most reduces latency?

- A** Emit all three Task calls in a single assistant response so the subagents execute concurrently
✓ correct
- B** Merge the three regional searches into one subagent that loops over regions internally
- C** Add a fixed iteration cap so the coordinator stops waiting after a set number of rounds
- D** Set tool_choice to 'any' so the coordinator selects tools faster

Why A is correct. Parallel subagents are launched by issuing multiple Task calls in one assistant response; because the three regional searches are independent, doing so collapses their latency to roughly the slowest single search. Merging into one looping subagent (B) removes the parallelism and the per-region differentiation. An iteration cap (C) does not address latency and risks dropping regions entirely. tool_choice 'any' (D) governs which tool the model picks, not whether subagents run concurrently.

Q24

D1 · Agentic

Task 1.2

Correct

Multi-Agent Research System

A newly configured coordinator answers research questions directly from its own

- A The subagents were registered at user scope while the coordinator runs at project scope
- B The coordinator's model is too small to plan multi-agent delegation

C The coordinator's allowedTools list omits 'Task', so it cannot spawn subagents
✓ correct

- D tool_choice is forced to a specific tool, blocking delegation

Why C is correct. To spawn subagents the coordinator's allowedTools must include 'Task'; when it is absent the coordinator has no delegation mechanism and silently answers on its own, exactly as observed. Scope mismatch (A) is not the gating mechanism for spawning and the subagents work standalone. Model size (B) does not explain a total absence of delegation attempts — the capability is simply not exposed. A forced tool_choice (D) would produce visible forced tool calls, not silent non-delegation.

Q25

D1 · Agentic

Task 1.6

Correct

Multi-Agent Research System

For open-ended briefs like 'assess the competitive landscape for X', the coordinator runs a fixed three-step chain (search then summarize then format). Reviewers note the reports consistently miss emerging subtopics that only surface mid-investigation. What orchestration change best fits this workload?

- A Add few-shot examples of good reports to the formatting step
- B Raise the temperature of the synthesis subagent to encourage broader

SP.



c Keep the fixed chain but add a second formatting pass

- D** Switch to dynamic decomposition so the coordinator spawns follow-up subtasks based on what earlier findings reveal

✓ correct

Why D is correct. Open-ended investigation calls for dynamic decomposition, where the coordinator spawns follow-up subtasks in response to what earlier findings reveal; a fixed chain suits only predictable, well-defined multi-aspect work and structurally cannot chase subtopics it did not pre-plan. Few-shot formatting examples (A) and a second formatting pass (C) improve presentation, not investigative coverage. Raising temperature (B) adds randomness, not the adaptive depth that surfaces emerging subtopics.

Q26

D1 · Agentic

Task 1.7

Correct

Multi-Agent Research System

After the shared search and document-analysis phase completes, stakeholders want two report variants from the same evidence — a one-page executive brief and a technical deep-dive — without re-running the costly searches. What is the best way to produce both?

- A Continue the same session, generate the brief, then ask for the deep-dive in the next turn

- B** Fork the session at the end of the research phase into two branches, one per report variant

✓ correct

- c Start two fresh sessions and re-run the search subagents for each

Why B is correct. `fork_session` creates divergent branches that share all prior state, so forking after the research phase lets each branch produce one variant from the same gathered evidence without re-searching. Continuing one session (A) entangles both outputs in a single history and biases the deep-dive on the already-generated brief. Two fresh sessions (C) wastefully re-run the expensive searches. A bigger context window (D) is unrelated to branching divergent outputs.

Q27

D1 · Agentic

Task 1.2

Correct

Multi-Agent Research System

Logs show the document-analysis subagent attempting to invoke the web-search subagent directly to fill gaps, producing routing errors and duplicated searches. How should the topology be corrected?

A Route all requests through the coordinator: subagents return gaps to it, and it dispatches any follow-up web searches

✓ correct

- B** Let subagents call each other but add a lock to prevent duplicate searches
- C** Merge the document-analysis and web-search subagents into one general subagent
- D** Give every subagent the full tool set so none needs to call another

Why A is correct. The correct topology is hub-and-spoke: subagents report findings and gaps back to the coordinator, which dispatches any follow-up work — peer-to-peer subagent calls are the defect. Adding a lock (B) legitimizes and entrenches the wrong topology. Merging the two subagents (C) collapses the intended specialization and differentiation. Giving every subagent the full tool set

Q28

D1 · Agentic

Task 1.7

Correct

Multi-Agent Research System

A weekly market report is produced by `--resume` of a named coordinator session. This week the report cites last week's prices as current, because the resumed session replays cached web-search tool results. What is the best fix?

- A Fork the prior session so the stale results are copied into a new branch
- B Keep using `--resume` but instruct the coordinator to ignore old tool outputs
- C Increase the context window so both old and new results coexist

D Start a fresh session, inject a concise summary of the prior report's conclusions, and re-run the searches

✓ correct

Why D is correct. When prior tool results are stale, the right move is to start fresh with an injected summary of the earlier conclusions and re-run the searches, so the report is grounded in current data rather than replayed cached results. Forking (A) copies the stale tool outputs into the new branch, preserving the problem. Prompt-based suppression under `--resume` (B) is unreliable and the outdated results still occupy context. A larger window (C) keeps the stale data resident, compounding the confusion.

Q29

D1 · Agentic

Task 1.3

Correct

Multi-Agent Research System

halting the workflow. How should loop termination be driven?

- A Add a minimum iteration count before the coordinator is allowed to stop
- B Use a stricter regex that also requires the word 'final'
- C** Continue the loop while stop_reason is tool_use (appending each tool result) and stop only on end_turn
✓ correct
- D Have a classifier judge whether the report is truly finished

Why C is correct. The agentic loop must be driven by the API's stop_reason — continue and append the tool result while stop_reason is tool_use, and terminate only on end_turn — never by parsing the model's natural-language text. A minimum iteration count (A) masks the bug with an arbitrary floor. A stricter regex (B) still parses assistant text and remains brittle. A classifier (D) is over-engineering for a deterministic termination signal the API already provides.

Q30

D2 · Tools & MCP

Task 2.3

Correct

Multi-Agent Research System

The synthesis subagent is initialized with the full registry of 18 tools shared across the system (web_search, fetch_url, parse_pdf, extract_tables, generate_chart, render_report, and others). Traces show that in 22% of runs it issues a fresh web_search or parse_pdf call instead of combining the findings the coordinator already passed in its prompt, adding 6-9 s of latency per run. What is the most effective fix?

- A** Restrict the synthesis subagent to the small set of tools its role actually needs,

- B Increase the synthesis subagent's context window so all prior findings fit and it stops re-fetching them
- C Add a system-prompt line instructing the synthesis subagent to never call `web_search` or `parse_pdf`
- D Force `tool_choice` to "any" so the model commits to exactly one deliberate tool call per turn

Why A is correct. Over-provisioning tools degrades selection: a role with 18 tools available will occasionally pick search or parse tools it should not touch. Scoping the synthesis subagent to only the tools its role needs removes the wrong options entirely and is the governing fix. A larger context window is irrelevant — the findings already fit; the model is choosing to re-fetch, not overflowing. A prompt prohibition is non-deterministic (the 22% proves prompt-only guardrails leak). `tool_choice` "any" makes it worse by mandating a tool call every turn when synthesis often needs none.

Q31

D2 · Tools & MCP

Task 2.3

Correct

Multi-Agent Research System

The document-analysis subagent must call `classify_document` before running any field extraction, because the extraction schema depends on the detected type. Relying on a prompt instruction, it skips classification in 9% of runs and extracts against the wrong schema. The coordinator invokes it once per document, and the required first step is always the same. What is the best fix?

- A Set `tool_choice` to "any" so the subagent is required to use some tool on the opening turn



On the first turn, set `tool_choice` to the forced form

- c Strengthen the system prompt with an emphatic "ALWAYS classify before extracting" plus two worked examples
- d Add an interception hook that inspects each extraction call and blocks it whenever no classification has run

Why B is correct. When a specific tool must run first deterministically, forcing `tool_choice` to that named tool on the opening turn is the direct API mechanism and guarantees `classify_document` executes before any extraction. `tool_choice` "any" forces a tool but not which one, so the model could still call an extraction tool. A stronger prompt is exactly the non-deterministic approach that already failed 9% of the time. An interception hook would work but is heavier cross-tool policy machinery for a single fixed first-step ordering that the forced `tool_choice` already guarantees at the API level.

Q32

D2 • Tools & MCP

Task 2.3

Incorrect

Multi-Agent Research System

The document-analysis subagent handles mixed inputs: some are PDFs requiring `parse_pdf`, and some are already plain-text snippets that need no tool at all. It is currently configured with `tool_choice` "any". Logs show it calling `parse_pdf` on plain-text inputs — returning garbage — in 18% of cases. What is the best fix?

- A Force `tool_choice` to the `parse_pdf` tool so parsing behavior is deterministic
- B Keep "any" but add a preprocessing stage that strips plain-text inputs out before dispatch



C Set `tool_choice` to "auto" so the model can respond directly when no tool is

D Add more few-shot examples of plain-text inputs to the subagent's prompt

X your answer

Why C is correct. tool_choice "any" forces at least one tool call on every input, so plain-text snippets get needlessly routed through parse_pdf. "auto" lets the model choose to use no tool when none applies, which is exactly the flexibility this mixed-input role needs. Forcing parse_pdf makes the wrong behavior mandatory. A preprocessing stage addresses the symptom at the wrong layer and adds a component when a one-line config change suffices. Few-shot examples cannot override the hard constraint that "any" mandates a tool call regardless.

Q33

D2 • Tools & MCP

Task 2.1

Correct

Multi-Agent Research System

The web-search subagent is equipped with two tools, search_web and search_academic, each carrying a one-line description ("search the web" / "search academic sources"). It routes queries to the wrong tool in 31% of cases — for example, sending peer-reviewed-literature queries to search_web. What is the best first step?

- A Consolidate the two tools into a single unified search tool
- B Add a separate router model that classifies each query and selects the tool programmatically
- C Set a forced tool_choice so the subagent always uses one designated search tool

D Expand and differentiate both tool descriptions — specifying input formats, example queries, source coverage, and explicit boundaries for when to use

Why D is correct. Tool descriptions are the primary tool-selection mechanism, so the first move is to enrich them: state input formats, give example queries, spell out source coverage, and draw explicit boundaries between the two. Consolidating destroys the useful distinction the two tools provide — here differentiating is the fix, not merging. A dedicated router model over-engineers a problem that a description edit resolves. A static forced `tool_choice` cannot work when the correct tool legitimately varies per query.

Q34

D5 · Context

Task 5.6

Correct

Multi-Agent Research System

The synthesis subagent merges findings from the web-search and document-analysis subagents into a unified narrative but drops the record of which source each claim came from. The report-generation subagent can then no longer attach citations, and 40% of the report's sentences ship uncited. What is the best fix?

- A Have the report-generation subagent re-run `web_search` to find a supporting source for each sentence
- B Add a citation post-processor that guesses the most likely source for each uncited sentence
- C** Preserve claim-to-source mappings through the synthesis step so each synthesized claim carries its originating source(s) into report generation
✓ correct
- D Instruct synthesis to insert "(source)" placeholders that a human fills in after the report is generated

Why C is correct. Provenance must be preserved through synthesis, not

based on. A guessing post-processor fabricates provenance. Placeholder markers defer the problem to humans and leave the report uncited by default. Carrying claim-source mappings end to end is the governing principle.

Q35

D5 • Context

Task 5.6

Correct

Multi-Agent Research System

Two sources report different market-size figures for the same market — \$4.2B (2021 data) versus \$6.8B (2023 data). The synthesis subagent silently picks \$6.8B and presents it as the single authoritative fact. Reviewers, seeing the discarded figure elsewhere, flag the report as unreliable. What is the best fix?

- A Always keep the larger or more recent figure and drop the other before synthesis
- B Average the two figures into a single blended number
- C Route every conflicting statistic to a human reviewer before synthesis can continue

D

Annotate both statistics with their source and collection/publication date, so the temporal difference is surfaced rather than misread as a contradiction

✓ correct

pipeline to preserve publication/collection dates precisely so temporal change is not mistaken for conflict. Silently keeping one figure (even the more recent one) or averaging destroys provenance and can be flatly wrong. Human routing is unnecessary overhead when the discrepancy is fully explained by date; escalation should be reserved for genuinely ambiguous conflicts, not every differing number.

Q36

D5 • Context

Task 5.1

Correct

Multi-Agent Research System

The coordinator passes each subagent's output forward verbatim: the web-search subagent's full scraped page bodies — roughly 38k tokens — flow straight into the synthesis subagent's prompt. Synthesis quality has dropped and it now misses findings buried in the middle of the passed-in content (lost-in-the-middle). What is the best fix?

A Trim each subagent's output to its relevant extracted findings before passing it downstream, and lead the synthesis prompt with a key-facts summary under section headers

✓ correct

- B** Move synthesis to a larger-context model so all 38k tokens fit comfortably
- C** Keep passing the raw pages but add a prompt instruction telling synthesis to focus on the important parts
- D** Have synthesis re-fetch and re-summarize each page itself before combining them

Why A is correct. Downstream context should be budgeted: trim verbose tool outputs to the relevant extracted fields before they accumulate, and counter lost-in-the-middle by placing the key summary first under clear section headers. A

to. Re-fetching duplicates work and re-introduces the same raw volume the trimming was meant to remove.

Q37

D1 · Agentic

Task 1.6

Correct

Developer Productivity with Claude

A coordinator agent is asked to produce a migration plan for a 400,000-line legacy billing monolith. The engineer hard-codes an upfront decomposition into five subagent tasks (billing, invoicing, tax, reporting, auth). Production runs show the final plan omits an entire 'dunning' subsystem and a currency-conversion layer that no subagent was ever assigned. What is the most effective decomposition strategy?

- A Raise the coordinator's maximum iteration count so it has more turns to notice the missing subsystems on its own

- B** Run an initial mapping pass that enumerates the repository's actual modules, then dynamically spawn one subagent per discovered component

✓ correct

- C Expand the fixed list from five to twelve subagent tasks based on the engineer's prior knowledge of the billing domain
- D Give the coordinator a larger context window so the entire monolith fits inside a single prompt

Why B is correct. For unfamiliar or legacy code you cannot know the true structure in advance, so any static decomposition guarantees blind spots — exactly the omitted 'dunning' and currency layers. The governing pattern is mapping-then-adaptive decomposition: discover the real module structure first, then decompose against what actually exists, which is why B is correct. A is wrong because more

confuses context size with decomposition; the failure is missed coverage, not insufficient window.

Q38

D1 · Agentic

Task 1.6

Correct

Developer Productivity with Claude

The coordinator spawns a subagent with a single Task call to trace how a payment webhook propagates through the `orders` module. Logs show the subagent re-runs Grep to rediscover the entrypoint the coordinator had already located two turns earlier, and reaches a conclusion that contradicts the coordinator's established finding. What is the root cause?

- A The subagent's allowedTools list is missing the Task tool, so it cannot delegate the trace further
- B The Message Batches API strips prior turns, so the multi-turn context was silently dropped
- C Subagents do not inherit the coordinator's conversation, so prior findings must be passed explicitly in the Task prompt**
✓ correct
- D tool_choice was set to "auto" instead of forcing the Grep tool for the trace

prior findings are passed in, the subagent rediscovers and can contradict them — making C correct. A is wrong because the Task tool governs spawning further subagents, not inheriting context. B is irrelevant: interactive delegation does not run through the Batches API. D is wrong because tool_choice controls which tool is invoked, not whether earlier conclusions are carried over.

Q39

D1 · Agentic

Task 1.7

Correct

Developer Productivity with Claude

An engineer runs `--resume` on a named exploration session from three days ago to continue a refactor. Since then a teammate moved several files and renamed a package. The resumed agent replays cached Read and Grep results from the old history and proposes edits against paths that no longer exist. What is the best way to continue the work?

- A Use `--resume` but manually enlarge the context window so more of the stale history is retained
- B `fork_session` from the three-day-old session to branch the work onto a new line
- C Re-run the same session with `/compact` to shrink the stale tool outputs before editing

D Start a fresh session and inject a short summary of the prior conclusions, discarding the stale tool results

✓ correct

Why D is correct. When prior tool results are stale, replaying them drives edits against a state of the world that no longer holds; the fix is to keep the conclusions but re-ground on the current tree — start fresh with an injected summary, so D is

SP.

moved files, so the wrong paths persist.

Developer Productivity with Claude

A security scan flags that the team's committed `.mcp.json` contains a hardcoded GitHub personal-access token used by an MCP server. The team wants the server config to stay shared and version-controlled, but each engineer (and CI) must supply their own credential without leaking it. What should they do?

- A Encrypt the `.mcp.json` file and commit the encrypted blob, decrypting it at session start.
- B** Reference the credential as `${GITHUB_TOKEN}` in `.mcp.json` so each environment supplies its own value from the environment at load time.
✓ correct
- C Move the server to user-scoped `~/.claude.json` so the token lives on a single trusted machine.
- D Store the token in the project `CLAUDE.md` so it loads with memory and is easy to rotate.

Why B is correct. MCP configuration supports `${ENV_VAR}` expansion precisely so secrets stay out of version control while the config itself remains shared: writing `${GITHUB_TOKEN}` lets each machine and CI job inject its own value at load time (B). Committing an encrypted blob (A) still ships a secret in the repo and adds key-management burden. Moving to user scope (C) sacrifices the requirement that the config stay shared and version-controlled. Putting the token in `CLAUDE.md` (D) commits the plaintext secret into VCS — the same leak, in a different file.

Q41

D2 · Tools & MCP

Task 2.4

Correct

Developer Productivity with Claude

The agent connects to a database MCP server to help engineers query a legacy

rarely. What is the best way to reduce this overhead?

- A Force tool_choice to "any" so the model commits to a productive tool sooner.
- B Raise the context window so more exploratory results fit without truncation.
- C Add a short-lived cache in front of the query tool so repeated probes return faster.

D Expose the schema catalog as an MCP resource the agent reads once upfront, instead of probing with repeated exploratory tool calls.

✓ correct

Why D is correct. MCP resources exist to expose content catalogs (schemas, indexes, file listings) that the agent can read directly, eliminating the exploratory tool calls used just to discover what is available (D). Because the schema is stable, a resource is read once and reused. tool_choice "any" (A) forces a tool but does nothing to remove the discovery calls. A larger window (B) accommodates the bloat rather than preventing it. A cache (C) speeds up the redundant probes but still incurs the round trips and token cost — the discovery pattern itself is the problem.

Q42

D2 • Tools & MCP

Task 2.4

Correct

Developer Productivity with Claude

The team added a code-intelligence MCP server that does semantic, cross-file symbol search over an indexed monorepo. In practice the agent almost always reaches for the built-in Grep tool instead, so it misses semantically related call sites that don't share literal text. The MCP tool's description currently reads only "Search the code index." What is the most effective first step?

SP.



agent prefers it over Grep.

✓ correct

- B Remove Grep from the agent's allowedTools so the MCP search tool is the only option.
- C Add a CLAUDE.md rule instructing the agent to never use Grep for code search.
- D Force tool_choice to the MCP search tool on every turn.

Why A is correct. Tool descriptions are the primary tool-selection mechanism, so a thin description ("Search the code index") loses to a well-understood built-in like Grep; enriching it with example queries and an explicit boundary (semantic vs literal matching) makes the agent prefer it where appropriate (A). Removing Grep entirely (B) is over-scoped — literal search is still the right tool for many tasks. A CLAUDE.md prohibition (C) fights good behavior with a blanket rule instead of clarifying when each tool wins. Forcing tool_choice every turn (D) breaks legitimate Grep and Read use and does not teach correct selection.

Q43

D2 · Tools & MCP

Task 2.5

Correct

Developer Productivity with Claude

The agent is asked to rename a shared constant throughout a generated client file. Its Edit call fails with a non-unique-match error because the target string appears on many lines that must all be updated consistently, and no single surrounding context makes one occurrence unique. What is the correct way to complete the change?

- A Switch to a Bash sed global substitution to bypass the Edit tool.

- c Delete the file and have the agent regenerate it from scratch with the new constant.

D Read the whole file, then Write it back with the modification, since Edit requires a unique match and this change spans repeated occurrences.

✓ correct

Why D is correct. Edit performs a targeted replacement on a unique match; when the anchor is inherently non-unique and the change spans many identical occurrences, the correct fallback is to Read the full file and Write it back with the edits applied (D). A sed substitution (A) sidesteps the built-in file tools and forfeits the reviewability and safety of Read/Write. Forcing tool_choice (B) does not resolve the ambiguity — Edit still cannot pick which occurrence to change. Regenerating the file (C) risks discarding unrelated hand-made changes and is far more destructive than needed.

Q44

D3 · Claude Code

Task 3.4

Correct

Developer Productivity with Claude

An engineer asks the tool to determine which of 30 services emits a particular Kafka event. Answering requires grepping and reading across dozens of files, producing thousands of lines of intermediate tool output. Done inline, this verbose discovery floods the main session's context and later steps degrade from lost-in-the-middle effects. What is the best approach?

- A Increase the main session's context window so all of the intermediate tool output fits

SP.



✓ correct

- c Turn on plan mode so the tool plans the searches before it runs them
- d Disable Grep and instead Read each candidate file directly in the main session

Why B is correct. Verbose, exploratory discovery belongs in an isolated Explore subagent that reports back only the concise answer, so the main context stays clean and free of attention dilution — making B correct. A is wrong because a larger window still accumulates and dilutes attention rather than preventing the flood. C plans the searches but does not reduce the volume of intermediate output they generate. D is slower and still pours every file's contents into the main context, worsening the problem.

Q45

D3 · Claude Code

Task 3.1

Correct

Developer Productivity with Claude

A senior engineer added the team's coding-convention instructions to `~/claude/CLAUDE.md`, and the tool follows them perfectly on her machine. A newly onboarded teammate clones the same repository, but the tool ignores those conventions for him. What is the correct fix?

- A Move the conventions into the project's `~/.claude/CLAUDE.md` (or root `CLAUDE.md`) checked into version control, since user-level memory is personal and not shared via VCS

✓ correct

- B Add the conventions to the teammate's `~/claude.json` MCP configuration so they load for him
- c Convert the conventions into a Skill so they load on demand whenever they

SP.



D Register the conventions in a `.claude/config.json` commands array so they sync with the repository

Why A is correct. `~/claude/CLAUDE.md` is user-scoped memory that never travels with the repository, which is exactly why a fresh clone does not pick it up; project-level `CLAUDE.md` is shared through VCS, so A is correct. B misuses MCP config, which defines tool servers rather than conventions and is itself personal. C makes the always-on conventions on-demand and still requires committing something to be shared. D references a `.claude/config.json` commands array that does not exist.

Q46

D3 • Claude Code

Task 3.6

Correct

Claude Code for Continuous Integration

A CI pipeline calls Claude Code to review each pull request, and a shell gate script greps stdout for the word "CRITICAL" to fail the build. The gate is flaky because Claude phrases severe findings differently each run — "severe issue," "major bug," "critical defect." What is the most effective fix?

A Add a post-processing library with a synonym dictionary that normalizes severity wording before the grep runs

B Invoke Claude with `--output-format json` plus a `--json-schema` so findings are emitted as machine-parseable structured fields the gate reads deterministically

✓ correct

C Instruct the prompt to "always use the exact word CRITICAL for severe issues" and rely on that wording

D Set temperature to 0 so the phrasing of findings stays stable between runs

and a `--json-schema` forces findings into typed fields (severity, file, line) the gate can evaluate deterministically, so key B is correct. A builds a brittle synonym-normalization layer that will always lag new phrasings and is unneeded once output is structured. C is a prompt-only instruction that offers no deterministic guarantee — the model can still deviate. D misunderstands temperature: even at 0 the output is prose, not a stable schema, and the gate remains fragile.

Q47

D3 · Claude Code

Task 3.6

Correct

Claude Code for Continuous Integration

In the pipeline, the same Claude Code session that generates an implementation also performs the automated review step. It consistently rates its own output clean, yet human reviewers keep finding real bugs it missed. What is the most effective change?

- ☒ A Run the review in a separate, independent Claude Code instance that shares no context with the generating session
✓ correct
- ☐ B Enable extended thinking on the generating session before it reviews its own code
- ☐ C Instruct the generating session to "review critically and assume the code contains bugs"
- ☐ D Lower the pass threshold so more borderline findings surface in the same session

Why A is correct. A session that produced code carries the same assumptions that caused its defects, so self-review systematically misses them. An independent instance with no shared context reviews the diff fresh and catches issues the

changes sensitivity, not perspective — the same session still fails to see the bugs it never flagged.

Q48

D3 • Claude Code

Task 3.6

Correct

Claude Code for Continuous Integration

Locally, the author's Claude generates tests that follow team conventions (naming, arrange-act-assert structure). In CI on a clean checkout, the generated tests ignore those conventions. The conventions are documented only in the author's `~/.claude/CLAUDE.md`. What is the best fix?

- A Pass the conventions as a `--system-prompt` argument in the CI command on every run
- B Add the conventions to the CI build server's `~/.claude/CLAUDE.md` so the runner picks them up
- C Reproduce the author's local machine environment inside the CI job

- ☒ D Move the conventions into the project's committed `.claude/CLAUDE.md` so every checkout — local and CI — loads them from version control

✓ correct

SP.



conventions into the project's committed `.claude/CLAUDE.md` makes them load for every clone, in CI and for every teammate, so key D is correct. A wires the guidance into a fragile command flag that lives outside the repo and drifts. B stashes instructions on one build server, still unversioned and invisible to teammates. C is disproportionate and does not address the real cause — the instructions simply are not in the shared project scope.

Q49

D3 · Claude Code

Task 3.6

Correct

Claude Code for Continuous Integration

The project's always-loaded `CLAUDE.md` has grown to embed a long, detailed code-review rubric. That rubric is needed only by the review job, but it is now injected into every CI job — test generation, doc updates — inflating token cost and diluting the model's focus on those tasks. What is the best fix?

- A Delete the rubric from `CLAUDE.md` and inline its full text into each review command invocation
- B Split `CLAUDE.md` into two files and symlink the right one into place per job
- C Keep the rubric in `CLAUDE.md` but wrap it in comment markers so it is visually separated

- D** Move the review rubric into an on-demand Skill under `.claude/skills` so only the review job loads it, keeping `CLAUDE.md` lean for always-on context

✓ correct

Why D is correct. `CLAUDE.md` is always loaded, so anything job-specific placed there taxes every unrelated run. A Skill loads on demand only when its job invokes it, so moving the rubric into `.claude/skills` keeps it available to the review job while leaving `CLAUDE.md` lean for the rest — key D is correct. A loses versioned reuse

Q50

D4 • Prompting

Task 4.1

Correct

Claude Code for Continuous Integration

The CI reviewer posts naming-convention and style comments, but developers report that roughly 60% of these particular comments are wrong or trivial. They have started ignoring the bot entirely — including its genuine security findings. What is the most effective response?

- A Add a single global confidence threshold so only comments the model rates above 0.8 are posted
- B** Temporarily disable the naming-convention category, then rewrite its prompt with explicit categorical rules before re-enabling it
✓ correct
- C Route every comment through a human moderation queue before it is posted to the pull request
- D Train a sentiment classifier on developer reactions and suppress whichever categories draw negative responses

Why B is correct. A single high-false-positive category erodes trust in the entire reviewer, so the governing principle is to remove the offending category from production and fix its prompt rather than paper over it. Temporarily disabling naming-convention comments stops the trust bleed immediately, and rewriting the category with explicit categorical criteria addresses the root cause before re-enabling it. A global confidence threshold is the wrong instrument — self-reported confidence is poorly calibrated and would also suppress good findings from healthy categories. A human moderation queue defeats the point of automation and does not fix the noisy prompt. A sentiment classifier is heavy over-engineering

SP.



Claude Code for Continuous Integration

The CI reviewer handles clear-cut defects well but is inconsistent on genuinely ambiguous cases — for example, whether a bare ``except:`` is acceptable in a one-off script versus a shared library. Enumerating every possible scenario in the prompt is impractical. What is the best approach?

- A

Add two to four few-shot examples of representative ambiguous cases, each with the reasoning behind its verdict

✓ correct
- B Write an exhaustive list in CLAUDE.md of every code pattern and its correct verdict
- C Fine-tune a dedicated model on the team's full history of past review decisions
- D Lower the bar so every borderline case is flagged and pushed to a human reviewer

Why A is correct. Few-shot examples are the right tool for ambiguous judgment because a small set of well-chosen cases with their reasoning lets the model generalize the underlying principle rather than memorize a lookup table. Two to four exemplars covering the representative ambiguous patterns teaches the script-versus-library distinction without enumerating everything. An exhaustive list is the enumeration approach the stem already rejects as impractical and it still will not cover unseen cases. Fine-tuning is disproportionate engineering for a prompt-level problem and is slow to iterate. Flagging every borderline case to humans abandons the goal of useful automated feedback and floods reviewers with noise.

SP.



session, reviews those tests for quality. The review almost never finds problems, yet audits show 20% of the generated tests have weak or tautological assertions. What explains this and what is the fix?

- A Enable extended thinking in the same session so the self-review reasons more carefully before approving
- B Have the same session review the tests three times and take a majority vote on each
- C Add more detailed quality criteria to the existing session's review prompt

D A session reviewing its own output is biased toward confirming it, so have a separate, independent Claude instance review the generated tests

✓ correct

Why D is correct. A session evaluating work it just produced is anchored to its own reasoning and tends to confirm rather than challenge it, which is why the self-review misses weak assertions the audit catches. An independent instance with no stake in the original generation reviews far more critically, so routing the tests to a fresh reviewer is the correct fix. Extended thinking makes the same biased session think longer about the same conclusion; it does not remove the conflict of interest. Voting across three passes of the same session compounds the shared bias instead of countering it. More detailed criteria help at the margin but cannot overcome a reviewer predisposed to approve its own output.

Q53

D4 • Prompting

Task 4.3

Correct

Structured Data Extraction

Production logs show that 8% of extraction responses fail the downstream `JSON.parse()` step because Claude wraps its output in markdown code fences or

- A Add a stern system-prompt line ("Return only valid JSON, no markdown, no commentary") and set temperature to 0 to make formatting deterministic.
- B** Define the target structure as a tool input_schema, force that tool via tool_choice, and read the structured JSON directly from the tool_use block.
✓ correct
- C Post-process every response with a regex that strips markdown fences and leading prose before calling JSON.parse().
- D Lower max_tokens so the model has no room to add explanatory prose around the JSON object.

Why B is correct. The governing principle is that tool_use with a JSON schema is the most reliable way to obtain structured output because the model returns a validated JSON object in the tool_use block, eliminating syntax and formatting errors entirely. Forcing the tool guarantees the structured path is taken. Prompt instructions plus temperature 0 (A) still rely on the model formatting free text and leave residual failures. Regex stripping (C) is a brittle patch that breaks on unexpected wrappers and does not address the root cause. Capping max_tokens (D) can truncate valid JSON and does not stop fence-wrapping.

Q54

D4 • Prompting

Task 4.3

Correct

Structured Data Extraction

After migrating to tool_use with a strict JSON schema, JSON parse failures dropped to zero. However, reconciliation shows 6% of extracted invoices have line-item amounts that do not sum to the stated invoice total. What does this reveal about schema-based structured output?

- A The schema is misconfigured; adding "additionalProperties": false will force

SP.



B tool_choice should be any rather than forced so the model can self-correct the arithmetic before returning.

- C** Schema enforcement removes syntax and type errors but not semantic errors, so a downstream arithmetic validation check must flag or trigger a retry on mismatches.

✓ correct

D max_tokens is too low, so the model omits line items; raising it will make the totals add up.

Why C is correct. The principle is that a JSON schema guarantees structural and type validity but cannot enforce semantic correctness such as line items summing to a total. The correct response is a downstream business-rule validation step that recomputes the sum and flags or retries mismatches. Option A confuses a structural constraint (additionalProperties) with an arithmetic invariant it cannot express. Option B misuses tool_choice, which controls tool selection, not numeric correctness. Option D misattributes a semantic discrepancy to a token limit; truncation would cause parse or completeness failures, not silent miscalculation.

Q55

D4 • Prompting

Task 4.3

Correct

Structured Data Extraction

For documents that lack a purchase-order number, audit logs show Claude inventing plausible-looking PO numbers about 11% of the time instead of signaling absence. Downstream systems then treat the fabricated values as real. What is the best schema-level fix?

- A Add a required confidence float to every field so downstream code can discard PO numbers below a threshold.
- B Switch tool_choice to "auto" so the model can decline to call the extraction

C Add a few-shot example containing a correctly formatted PO number to anchor the expected pattern.

- D** Make the `po_number` field nullable/optional in the schema and instruct the model to return null when the value is not present in the source.

✓ correct

Why D is correct. The principle is that making a field nullable or optional gives the model a sanctioned way to represent absence, so it returns null instead of fabricating a value to satisfy a required field. Pairing this with an explicit instruction to emit null when the data is absent directly addresses the hallucination. A confidence score (A) is poorly calibrated and does not stop the fabrication, only tries to filter it afterward. Setting `tool_choice` to "auto" (B) risks skipping extraction entirely for otherwise-complete documents. A few-shot example of a valid PO (C) reinforces the format but gives no path for the missing case, which is the actual problem.

Q56

D4 • Prompting

Task 4.3

Correct

Structured Data Extraction

The extraction schema uses an enum with six known values for `document_type`. A new vendor begins sending "credit memo" documents, which are not in the enum, and the model force-fits them to "invoice", corrupting downstream routing. What is the best design change?

- A** Add an "other" value to the enum plus an optional `document_type_detail` string so novel types are captured verbatim instead of misclassified.

✓ correct

- B** Remove the enum constraint entirely and let the model emit any free-text string for `document_type`.

- D Brainstorm and add every conceivable document type to the enum so no future document falls outside it.

Why A is correct. The principle is designing enums for extensibility with an "other" (or "unclear") escape value plus a free-text detail field, so unanticipated categories are surfaced rather than silently forced into a wrong bucket. This keeps the controlled vocabulary intact while capturing novel cases for review. Dropping the enum (B) sacrifices the validation and normalization that make the field useful downstream. A fine-tuned classifier (C) is heavy over-engineering when a schema change solves the problem. Enumerating every possible type (D) is impossible to complete and still breaks on the next truly novel document.

Q57

D4 • Prompting

Task 4.4

Correct

Structured Data Extraction

Extractions pass the JSON schema but sometimes violate a business rule (invoice_date later than due_date). The current recovery discards the result and re-runs the identical prompt from scratch; 40% of retries still fail the rule. What is the most effective retry strategy?

- A Re-run the identical prompt up to five times and accept the first result that happens to pass the rule.

B Append the previous failed extraction and the specific validation error messages to a follow-up message, asking the model to correct only the offending fields.

✓ correct

- C Lower temperature to 0 and re-run the original prompt exactly once.

Why B is correct. The validation-retry principle is to feed the model its own failed output plus the exact validation errors, so it can correct the specific fields rather than re-deriving everything blindly. This targeted feedback is far more likely to resolve a rule violation than repetition. Re-running the same prompt (A, C) provides no new information and mostly reproduces the same error. The Batch API (D) is for high-volume latency-tolerant work, not a corrective loop, and it does nothing to give the model the failure context it needs to fix the fields.

Q58

D4 · Prompting

Task 4.4

Correct

Structured Data Extraction

For scanned documents where the tax ID is physically cut off at the page edge and simply absent from the source text, the validation-retry loop retries five times and still fails, burning tokens on every attempt. What is the correct understanding of this situation?

- A Raise the retry ceiling to ten; with enough attempts the model will eventually recover the tax ID.
- B Enable extended thinking so the model reasons more deeply and reconstructs the missing tax ID.

C Retries can fix format and structural errors but cannot recover information absent from the source, so detect the absence, return null, and route the document to human review instead of retrying.

✓ correct

- D Switch tool_choice to "any" so the model can select a different extraction tool that will locate the tax ID.

document. Recognizing absence, returning null, and escalating to a human is the correct terminal behavior, and it stops the wasteful token spend. Increasing retries (A) cannot recover data that was never in the input. Extended thinking (B) improves reasoning over available evidence but cannot invent an absent value without fabricating it. Changing tool_choice (D) alters which tool runs, not what text exists on the truncated page.

Q59

D4 • Prompting

Task 4.5

Correct

Structured Data Extraction

A 50,000-document extraction batch completes and 300 requests returned errors. The team needs to reprocess only the failures and reliably match every result back to its source document. What is the correct approach?

A Assign each request a custom_id mapping to its source document, correlate results by custom_id, and submit a new batch containing only the 300 failed custom_ids.

✓ correct

- B** Resubmit the entire 50,000-document batch, since batches are atomic and cannot be reprocessed partially.
- C** Rely on result ordering, since the Batch API guarantees that output order matches input order.
- D** Convert the 300 failures to synchronous calls and use response timestamps to align each result with its source document.

Why A is correct. The principle is that each Batch API request carries a caller-supplied custom_id, which is the correlation key for matching results back to sources and for selectively resubmitting only the failed items. Reprocessing the

matching (D) is fragile and unnecessary when a deterministic custom_id mapping is available.

Q60

D4 · Prompting

Task 4.3

Incorrect

Structured Data Extraction

The pipeline handles six document types, each with its own extraction tool. Even when the type is already known from the intake folder that routed the document, the caller still sends all six tools with tool_choice "auto", and in 9% of cases the model invokes the wrong extractor. What is the best fix?

- A Keep tool_choice "auto" but add sentiment analysis of the document to help the model decide which extractor fits.

- B** When the document type is already known from routing, force the specific extractor with tool_choice {"type": "tool", "name": "extract_invoice"} rather than leaving selection to "auto".

✓ correct

- C** Set tool_choice to "any" so the model is at least required to call some extraction tool.

✗ your answer

- D Merge all six extraction tools into one large tool with a combined schema covering every document type.

Why B is correct. The principle is that when the correct tool is already determined by upstream logic, you should force it with tool_choice {"type": "tool", "name": "..."} instead of asking the model to re-decide, which removes the selection error entirely. "auto" (A) reintroduces the very choice that is failing, and sentiment analysis is irrelevant over-engineering. "any" (C) still lets the model pick the wrong

[↶ Retake exam](#)

YOU PASSED THE MOCK. WHAT ABOUT THE TEAM?

The exam measures what you know. The Enterprise Agentic AI Masterclass is two days that leave your team with its own agent and code ready to ship. Max 12 people.

[See the workshop →](#)[Book a call →](#)

Scaled score is an approximation: $100 + 900 \times (\text{correct} \div \text{total})$, with the 720 pass line matching the real exam. Use it as a directional signal, not a guarantee.

Unofficial practice material — not affiliated with or endorsed by Anthropic. All questions are original and modeled on the publicly available Claude Certified Architect – Foundations exam guide.

For a team lead

This test measures one person. A team is a separate question.

If you landed here because you are weighing up training for a whole team, everything you need to forward is below — no need to read the offer page. This section has its own link, so you can send it to whoever approves the budget.

SP.

laptop.

WHAT YOU NEED BEFORE THE DATE

The repository we will work on, a written description of the stack and the goal, and accounts with tool access on company hardware and the company network.

PROGRAMME

Two days. The first on using agents in day-to-day work with code, the second on building your own. Six hands-on modules, no theory off slides.

FORMATS AND PRICES

Live online from \$4,800, on-site from \$6,400 with travel included. For a team of up to 12 people, over two days. Dates are agreed individually.

WHAT IT IS NOT

It is not a preparation course for the Claude Certified Architect exam. This test and the explanations to all 60 questions are free and stay free. The workshop teaches production work, not exam technique.

See the full programme and the form →

The enquiry form sits on the workshop page — I answer the same question you type into it.

SP.

ex-CTO · AI Strategy · Implementation



SERVICES

Consulting

Implementation

Workshops

Speaking

AI readiness assessment

MORE

About

Blog

Agentic Engineering Radar

Case studies

Framework

Consultant Program

Workflow Studio



EU AI Act compliant



GDPR compliant



NIST AI RMF

© 2026 Szymon Paluch. AI without the bullshit.